Mini-Projects

Mini Project 2

Mini-Project 2 · Released Sep 16, 2026 · Due Oct 15, 2026

> Build a weather client and a networked mastermind game over raw sockets, and implement copy-on-write fork and alarms in xv6.

Before you start

This is an individual assignment.

Deadlines

Submit both parts of the assignemt by **October 15 11:59 pm**. No extensions will be provided.

Additionally, doubts asked on the doubt document past **11:59 PM, October 10** will not be answered.

Submission

You will be provided a template repository on `code.iiit.ac.in`. This repo will contain the base files that you will need to work on. Any other Github or Forked Repositories will not be considered as submission. Make sure to work on the private repository allocated to you under the `osn` organisation.

You will need to commit iteratively and coherently as you go. **You will have to push after every single commit. (Within 24 hours of each commit)**

Your commit message must clearly describe the specific functionality implemented in that particular commit. For example, in `tempest`, if you have implemented parsing of HTTP status line and headers, your commit message can be `tempest: parse HTTP status line and headers`.

Making a bunch of big commits right before the deadline or commits without a clear purpose and random names will result in penalization.

You can refer to the resource on git for more details.

Your repository has the following structure:

```
│   ├── tempest/
│   ├── mastermind/
├── xv6/
│   ├── user/
│   ├── kernel/
│   ├── Makefile
│   └── ...
├── ai-usage.md
└── readme.md
```

The `readme.md` must contain build instructions, run instructions, assumptions you made, and known bugs.

AI Usage

You are **NOT** allowed to use AI agents to generate code (or the report) for any part of this assignment. You may use AI chatbots for understanding concepts and debugging errors.

Submit the prompts used in a document named `ai-usage.md`. You must submit only links to your chat; no screenshots are allowed. Faking links or failing to submit the AI usage document will lead to strict penalties.

Grading

You will be graded based on both your implementation and an in-person evaluation with a TA. You are expected to understand the code that you submit.

Doubt Doc

Doubts | Answers

> All the best! Have fun :)

Networking [55]

General Requirements

- Break the project down into multiple `.c` and `.h` files based on functionality. Monolithic code will be penalized.

- Compile with the same flags as Mini Project 1:

```
gcc -std=c23 \
  -D_POSIX_C_SOURCE=200809L \
  -D_XOPEN_SOURCE=700 \
  -Wall -Wextra -Werror \
  -Wno-unused-parameter \
  your_file.c
```

Copy

- You may link `-lm` for maths. Nothing else.

Allowed networking API: Only the raw POSIX sockets interface: `socket`, `bind`, `listen`, `accept`, `connect`, `send`, `sendto`, `recv`, `recvfrom`, `setsockopt`, `getsockopt`, `getaddrinfo`, `shutdown`, `close`, and `poll`, `select` or `epoll`.

Banned: Any library that implements HTTP, a reliable datagram protocol, or message framing for you. This includes `libcurl`, `libevent`, `libuv`, `ZeroMQ`, `ENet` and anything similar. Shelling out to `curl`, `wget` or `nc` is also banned. Using any of these gets a 0 for that part.

References

- `man 2` pages for `socket`, `bind`, `connect`, `send`, `recv`, `setsockopt`, `poll`
- `man 3` pages for `getaddrinfo`, `inet_ntop`
- Beej's Guide to Network Programming
- RFC 7230 sections 3.1 to 3.3, for HTTP message syntax and chunked encoding
- RFC 3550 section 6.4.1, for the interarrival jitter formula
- Shivang Srivastava's article on how to write an HTTP request from scratch

Important

Please do not test your networking assignment, especially UDP discovery, on IIIT-H Network (LAN and WiFi). Badly written code (like a runaway loop) will quickly overwhelm the IIIT network with packets causing overload and potentially a Denial Of Service for the network. Rather connect your laptop (and friend's if testing `mastermind`) to your mobile hotspot and then test your program.

DON'T: Test your program on IIIT Wifi or LAN.

Part A: tempest [15]

Feena has been tortured by her RA assignment and hasn't seen the sun in three days. Now that she is planning to go out after finally finishing her assignment, she is a bit unsure about the whether it actually is sunny outside. Help her find the current weather for her desired location.

For this section, you are required to send HTTP requests to get Feena the latest weather report using the service `http://wttr.is`.

To do this, you shall use HTTP/1.1 connection to the endpoint:

```
http://wttr.is/<city_name>?0T
```

Copy

where `<city_name>` is the location for which you want to fetch the weather for. It must be URL-encoded.

Examples:

```
$ curl http://wttr.is/Hyderabad?0T

Weather report: Hyderabad

                Overcast
       .--.     +22(25) °C
    .-(    ).   ↙ 4 km/h
   (___.__)__)  10 km
                0.0 mm
```

Copy

```
$ curl http://wttr.is/New%20York?0T

Weather report: New York

                Overcast
       .--.     +15(9) °C
    .-(    ).   ↓ 25 km/h
   (___.__)__)  10 km
                0.0 mm
```

Copy

Requirements:

2. The program must accept the city name for which you wish to view the weather as its argument. Example: `$ tempest Hyderabad`
3. If more than one argument is specified by the user, print `tempest: too many arguments`.
4. If an invalid location is provided as an argument, print `tempest: invalid location`.
5. Short reads and writes must be correctly handled on the socket.
6. Implement a 10 second timeout for requests. If a request takes longer than 10 seconds to complete, it must be terminated (If you change the duration of this timeout, mention the same in your `readme.md`).
7. You must not print the HTTP response headers to the terminal when no flag is specified.
8. Implement a `--raw` flag that causes the program to print the exact response received from the endpoint, including the HTTP response headers. The flag must appear after the argument provided by the user.

Example:

```
$ tempest Rotterdam --raw
> <YOUR HTTP REQUEST HERE>
> ...
> ...
> ...
< HTTP/1.1 200 OK
< Access-Control-Allow-Origin: *
< Cache-Control: public, max-age=600
< Content-Type: text/plain; charset=utf-8
< Date: Sun, 13 Sep 2026 10:36:33 GMT
< Content-Length: 173
<
Weather report: Rotterdam

   _`/"".-.     Light rain shower
    ,\_(   ).   19 °C
     /(___(__)  → 14 km/h
       ‘ ‘ ‘ ‘  10 km
      ‘ ‘ ‘ ‘   0.9 mm
```

10. You must understand the HTTP request and response headers used by your program, and what each of the fields in them do (the ones that you have used in this assignment, at the very least)

1. RFC 9112, Section 2
2. RFC 9112, Section 3

> Fun fact: tempest comes from the latin word *tempestas*, meaning “weather” :)

Part B: mastermind [40]

Mastermind is a popular guessing game played between two players. One player thinks of a sequence (usually colors, but we will consider a string of digits for simplicity), while the other tries to guess it within 12 attempts. The guessing player suggests one sequence at a time and the player responding gives feedback about the sequence.

> For ease of explanation, consider the player guessing (codebreaker) as `Tatva` and player responding (mastermind) as `Feena`.

Rules

1) Each sequence has 5 digits ranging from 0 and 9. 2) Feena sets a `master sequence` of 5 digits. The positions of the digits in this sequence define the correct sequence. 3) There are 12 attempts for Tatva to identify the correct sequence. 4) Each sequence attempted by Tatva is an attempted sequence. 5) The original game uses white, red and no pegs for feedback, we shall be using `o`, `x` and `-` respectively. 6) The following feedback is given for each attempt by Feena to Tatva:

- For each digit that is there in both the sequences and in the correct position in the attempted sequence, the feedback is given as a red peg (`x`).
- For each digit that is there in both the sequences but is in the wrong position in the attempted sequence, the feedback is given as a white peg (`o`).
- For each digit that is present in the attempted sequence but not the master sequence, feedback is given as (`-`). 7) The feedback is given out of order, i.e., you do not need to give a red peg (`x`) in the same position as the digit for which it is meant. 8) Each digit may be used multiple times in the code, in which case the feedback given will be for individual digits, i.e., if the master sequence has 4 counts of the digit 2, then 4 separate feedback pegs will be given for each of those digits depending on their position.

References:

- Here is a link to a video of the rules for further understanding (Note: this video uses a smaller 10x4 board; black pegs instead of red; and doesn’t allow for repeats, but the core idea remains the same).

Example:

Feena's (mastermind) view:

```
12345  -----
65731  xo---
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
-----
67676
```

Tatva's (codebreaker) view:

```
12345  -----
65731  xo---
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
*****  *****
-----
*****
```

- Both the players must run the same program, there are no separate instances of client and server programs.
- Your program must use UDP discovery (see below) to find another instance of the program running on the same Local Area Network (LAN).

the purpose and are clearly defined in both a `.h` file and the `readme.md`

- You are not required to implement threads, look at non-blocking sockets and related functions.

Player discovery [8 marks]

UDP broadcast allows devices to identify and find other devices over the same network by sending UDP packets to the broadcast address. These packets can be received by other players on the same network, allowing them to discover which other players are currently online.

Requirements:

1. Implement a UDP broadcast-based discovery system that allow Tatva and Feena to discover each other when they are over the same LAN.
2. Include the player's name and port number (used to listen for connections) in the broadcast packet. Do not include the player's IP address in the payload; instead, use the packet's source IP itself.
3. The broadcast frequency may be set to once every 2 seconds. (If you are changing this frequency, mention the same in your `readme.md`)
4. Use of a magic/identifier packet to determine whether the packet belongs to our program or another program on the network is necessary. You must ignore packets from other programs. You are free to choose the magic packet, as long as it is 4 bytes and documented in the your `readme.md`.

Starting the game [2 marks]

On startup, the game asks the player to enter their name. The **program starts listening** for connections on the port that is **to be sent in the payload**, and then starts the UDP broadcast-based discovery.

Both Tatva and Feena start the game independently on their systems. Once started, each of them can discover other players on the same network using UDP-broadcast discovery.

The game should display the discovered players in the following format. For example, if Tatva and ristiavdaatnso are online, Feena should see:

```
Players Online:
ID       Name          IP Addr      Port    Last Seen
1    Tatva           10.2.35.123  8080    2 s. ago
2    ristiavdaatnso  10.2.35.125  8000    1 s. ago
```

Copy

This list should be updated every time a new broadcast packet is recieved. An entry that has not been refreshed by a new broadcast for 5 seconds should be removed from the list.

Use `challenge <ID>` to challenge a player to a game.

If Feena wishes to play a match against Tatva, she can enter:

```
> challenge 1
```

Copy

This sends a TCP connection request to Tatva at `10.2.35.123:8080`. Tatva will then be shown a request which he can choose to accept (by typing `yes`) or reject (by typing `no`).

Typing `no` will return to the UDP discovery mode, while typing `yes` will start the game (and setup a persistent TCP socket).

Gameplay [15 marks]

> The one who accepts the challenge becomes the codebreaker (person guessing) while the one who initiates becomes the mastermind (person responding)

After accepting the request, Tatva becomes the codebreaker while Feena becomes the mastermind. Feena is shown a prompt to enter the `master sequence`.

Once the input sequence is received, perform validation on the input sequence to ensure: 1. Length of the sequence is exactly 5 digits 2. All 5 characters are digits

This validation must be done for every subsequent attempted sequence as well.

Feedback must also be validated to ensure that its length is 5 characters, and that the sequence only contains valid characters (`o`, `x`, `-`).

Tatva then makes the first attempt. On pressing Enter, the sequence is sent over the network using the TCP socket established earlier.

Feena receives the attempt, assigns the appropriate feedback, and then presses Enter to send it back to Tatva. The feedback sent will be displayed on Tatva's board (ref. earlier example).

The feedback received by Tatva should be coloured: `o` will be yellow, `x` will be green and `-` will be red. You may use ANSI escape sequences for achieving this.

based on the attempts and feedback received.

Tatva and Feena should have the same view of the board except that Feena can see the master sequence and Tatva sees `*****` in its place (refer the earlier example of the board to see where the master sequence is shown)

Requirements:

1. Do not send over the master sequence to the codebreaker until the game has ended.
2. The TCP Socket created at the beginning must be kept persistently open throughout the game providing a bi-directional communication channel between both players over the same socket. This allows the players to send or receive messages at any point during the game.

Ending the game

The game ends when either of the following situations happen:

1. Tatva runs out of attempts
2. Tatva is able to find the correct sequence in less than 12 attempts

In both cases, Feena's program sends over the master sequence to Tatva's program and closes the TCP socket. Tatva's program then displays the master sequence on his board. Pressing enter after this takes the players back to homepage (where the UDP discovery is shown).

Failure Management

If either player closes their application or gets disconnected, the other player's program must detect the disconnection and display:

`<Player_name> disconnected. Press enter to go home.`

Pressing enter takes the player back to the homepage, where UDP discovery is displayed again.

This must be implemented for both TCP (normal game) and UDP (cost cutting). Your `readme.md` must describe how connections are detected for both the protocols.

> Think about the socket and its persistence. How can you use this to your advantage?

Cost Cutting [15 marks]

The institute has disallowed TCP packets on the LAN network to save on cost and bandwidth. This has made it impossible for Feena and Tatva to play the game, since it uses TCP for state transfer

To save them from this gloomy situation, Rishavdaathso suggested replacing TCP with UDP for communication between the players.

When your program is run with the `--cost-cutting` flag, it must use UDP instead of TCP for all state transfers beteen players.

Requirements:

You need to implement state transfer (i.e. the persistent TCP socket) using UDP. Since UDP does not guarantee ordering of packets or reliable delivery, you need to implement the following on your own to provide a seamless experience during gameplay:

1. The sender must divide text data into fixed-size chunks. Each chunk must have a sequence number, sent using a `struct`. The sender must also communicate the total number of chunks. Once all chunks are received, the receiver must reorder them using their sequence numbers, aggregate them, and display the complete text.
2. The receiver must send an ACK for every received chunk, referencing its sequence number. If the sender does not receive an ACK within **0.1 seconds**, it must retransmit the corresponding chunk. The sender must not wait for an ACK before transmitting subsequent chunks.

Logging [5 marks]

You must report all the implementation details and assumptions taken in your `readme.md`.

Your program must support logging using the `--log` flag. When enabled, every event recieved or sent by the program must be recorded in `log.txt` (append, do not overwrite) in the same directory as the program. Use the following piece of code to generate the timestamp for the log.

Copy

```
#include <stdio.h>
#include <sys/time.h>
#include <time.h>

// Inside your logging function

char time_buffer[30];
struct timeval tv;
time_t curtime;

gettimeofday(&tv, NULL);
curtime = tv.tv_sec;

// Format the time part
strftime(time_buffer, 30, "%Y-%m-%d %H:%M:%S", localtime(&curtime));
```

You now have a completely functional game of mastermind that you can play when you're procastinating doing assignments!!

xv6 [90]

For this part of the mini project, you will implement copy-on-write fork, alarms and alarm handlers.

It is recommended to not work on the CoW fork and alarms at the same time since both modify the trap handler. We would recommend working on two different git branches and rebasing once you're done with your work, so that all the alarm commits and the CoW fork commits are grouped

Copy-on-write fork [50]

xv6's `fork()` calls `uvmcopy()`, which allocates a fresh physical page for every page of the parent process and copies it to the child process. Most of that work is usually wasted since a common pattern is a `fork()` immediately followed by `exec()`, which throws the whole copy away.

Why copy pages that won't be written to? To fix this problem, you will implement **copy-on-write fork** which makes multiple virtual pages point to the same physical page and **defers copying a page until it is written to**. Two advantages of this is that read-only pages are never copied, and that writable pages aren't copied unless they are actually needed.

Your submission is correct if both `cowtest` and `usertests -q` pass.

Approach

- `kalloc` and `kfree` are responsible for allocating and freeing kernel pages. When you are making multiple virtual pages point to the same physical page, you now need to take care to not free physical pages which are being pointed to by any virtual pages. For this, implement **reference counting** for the physical pages. **[10 marks]**
- `uvmcopy` copies memory from the parent process to the child process. Modify it so that the child's virtual pages point to the parent's physical pages. **[15 marks]**

  - Mark the page CoW (some pages are simply read-only. don't go around duplicating every such page). xv6's paging hardware reserves 2 bits of a PTE for software, use that to your advantage.
  - Allocate a new physical page and make the faulting virtual page point to it.
- You will also need to modify the `copyout` function to handle copy-on-write pages. **[10 marks]**

> Alternate approaches are welcome, but will be graded in a binary fashion, i.e., you will get full marks if your solution works and 0 if it doesn't.

Alarms [40]

For this part of the mini-project, you will allow processes to register an **alarm handler**: a function defined in a process' code that is run every `n` ticks of the CPU. This is done through the `sigalarm` system call which registers the handler and the period `n`. The `sigreturn` system call must be called at the end of the handler for the process to return to regular execution context.

Your submission is correct if both `alarmtest` and `usertests -q` pass.

System calls

```
int sigalarm(int ticks, void (*handler)());
int sigreturn(void);
```

Copy

`sigalarm(n, fn)` asks the kernel to call `fn` after every `n` ticks of CPU time this process consumes. `fn` ends by calling `sigreturn()`, which resumes the interrupted code as if nothing had happened. `sigalarm(0, 0)` cancels a pending alarm.

Requirements

- `sigalarm` must return 0 if the handler was registered successfully and a non-zero value otherwise (by the way, null pointers are very much valid pointers to data in xv6).
- `sigreturn` must return the return value of the alarm handler.
- Only invoke the function if the process has a timer outstanding (note that the address of the user's alarm might be 0).
- You must ensure that when the alarm handler is done, control returns to the instruction at which the user program was originally interrupted by the timer interrupt.
- The user program must continue undisturbed after the alarm.

- User alarm handlers are required to call `sigreturn` syscall when they finish.
- If a handler hasn't returned yet, the kernel should not call it again.

Approach

- Modify `proc.h` and `allocproc.c` to declare and initialize fields that are required to facilitate alarm handlers. **[10 marks]**
- Implement the sigalarm and sigreturn system calls. The `sigalarm` call registers the handler (you could stord the handler's address in the `proc` struct) and the sigreturn call must restore the state of the process. **[15 marks]**
- Modify the timer interrupt part in `usertrap` to call the alarm handler if a process' `n` ticks are up. Make sure to save relevant parts of the process state! **[15 marks]**

> Alternate approaches are welcome, but will be graded in a binary fashion, i.e., you will get full marks if your solution works and 0 if it doesn't.

‹ Previous
Mini Project 1